API SECURITY RISK ANALYSIS REPORT

JSONPlaceholder REST API Security Assessment

Prepared by: ANGELO BOSCO SAMJELA

CIN: FIT/APR26/CS7778

Cyber Security Intern | Future Interns

Institute of Finance Management (IFM), Dar es Salaam, Tanzania

Date: May 06, 2026

TABLE OF CONTENTS

1. INTRODUCTION 3
2. OBJECTIVES 4
3.SCOPE AND METHODOLOGY 5
3.1 Target API Overview 5
3.2 Tools Used 5
3.3 Testing Methodology 5
4. API ENDPOINT TESTING RESULTS 7
4.1 Test 1: Unauthenticated Access to All User Records (GET /users) 7
4.2 Test 2: Unauthenticated Access to Individual User PII (GET /users/1) 9
4.3 Test 3: Unauthenticated Data Creation (POST /posts) 10
4.4 Test 4: Unauthenticated Data Modification (PUT /posts/1) 11
4.5 Test 5: Unauthenticated Data Deletion (DELETE /posts/1) 12
4.6 Test 6: Response Header Security Analysis 13
4.7 Test 7: Invalid Resource Handling (GET /users/999999) 15
5. SECURITY RISK FINDINGS 17
6. RISK SUMMARY 18
7. BUSSINESS IMPACT ANALYSIS 18
8. REMEDIATION RECOMMENDATIONS 19
9. WHAT THIS MEANS IN SIMPLE TERMS 20
10.LONG-TERM SECURITY RECOMMENDATIONS 21
11.CONCLUSION 22

1. INTRODUCTION

Application Programming Interfaces commonly known as APIs are the invisible backbone of modern digital services. They allow applications to communicate with each other, share data, and execute actions on behalf of users. When you use a banking app to check your balance, a food delivery platform to place an order, or a social media app to post an update, an API is working behind the scenes to make it happen.

Despite their critical role in today's digital infrastructure, APIs are frequently the weakest link in an organization's security posture. According to the OWASP API Security Top 10, broken authentication, excessive data exposure, and lack of authorization controls are among the most common and most dangerous API vulnerabilities in the wild. In 2023 alone, API-related breaches cost organizations billions of dollars globally.

This report presents a structured security risk analysis conducted against the JSONPlaceholder REST API a publicly available, open test API hosted at jsonplaceholder.typicode.com. The API was selected specifically because it is designed to simulate a real-world REST API environment, making it an ideal and legal target for security testing and analysis.

It is important to note that JSONPlaceholder is intentionally built without security controls for demonstration purposes. The vulnerabilities identified in this report are real and measurable, and they directly mirror the types of API security failures found in production systems worldwide. This makes the analysis highly relevant from both a training and a real-world applicability standpoint.

2. OBJECTIVES

The following objectives guided the scope and execution of this security assessment:

1. Identify API endpoints that expose sensitive data without requiring any form of authentication or authorization.
2. Test all major HTTP methods GET, POST, PUT, DELETE to determine whether write operations can be performed without credentials.
3. Analyze HTTP response headers for the presence or absence of critical security controls such as Content Security Policy, X-Frame-Options, and CORS configuration.
4. Evaluate the API's error handling behaviour when invalid or out-of-range resource identifiers are requested.
5. Classify all identified security risks using the OWASP API Security Top 10 framework and assign appropriate risk ratings.
6. Produce a professional security risk report that is clear and actionable for both technical developers and non-technical business stakeholders.
7. Provide concrete, prioritized remediation recommendations to address each identified vulnerability.

3.SCOPE AND METHODOLOGY

3.1 Target API Overview

Target API Name	JSONPlaceholder
Base URL	https://jsonplaceholder.typicode.com
API Type	RESTful API (JSON responses)
Purpose	Free public test API for prototyping and security research
Hosting	Cloudflare CDN / Heroku Router (Express.js backend)
Available Resources	Users, Posts, Comments, Albums, Photos, Todos
Authentication	None the API operates with no authentication mechanism
Assessment Date	May 07, 2026
Legal Status	Publicly available and authorised for security testing

3.2 Tools Used

No	Tool	Purpose	Platform
1	Postman	Send HTTP requests (GET, POST, PUT, DELETE) and inspect responses, headers, and status codes	Windows desktop application
2	Browser DevTools	Supplementary header inspection and network traffic analysis	Windows Firefox / Chrome
3	OWASP API Security Top 10	Reference framework for classifying and rating identified API vulnerabilities	owasp.org

3.3 Testing Methodology

The assessment followed a seven-step black-box testing approach, treating the API as an external attacker would with no prior knowledge of internal code, infrastructure, or credentials:

Step	Phase	Description
1	**Reconnaissance**	Identified available API endpoints and resource types by reviewing the JSONPlaceholder documentation.
2	**Unauthenticated Read Testing**	Sent GET requests to user and post endpoints without any authentication headers or tokens to test for broken access control.
3	**Unauthenticated Write Testing**	Sent POST, PUT, and DELETE requests without credentials to test whether data creation, modification, and deletion are restricted.
4	**Response Header Analysis**	Inspected all HTTP response headers returned by the API to identify missing or misconfigured security headers.
5	**Error Handling Testing**	Requested non-existent resources using out-of-range IDs to evaluate how the API handles invalid inputs.
6	**Risk Classification**	Mapped all findings to the OWASP API Security Top 10 and assigned severity ratings based on exploitability and business impact.
7	**Reporting**	Compiled all findings, evidence screenshots, risk ratings, and remediation recommendations into this structured report.

4. API ENDPOINT TESTING RESULTS

4.1 Test 1: Unauthenticated Access to All User Records (GET /users)

HTTP Method	GET
Endpoint	https://jsonplaceholder.typicode.com/users
Authentication	None- no token, key, or credential provided
HTTP Status	200 OK
Response Size	2.94 KB
Response Time	1.59 seconds
OWASP Category	API1:2023 Broken Object Level Authorization / API3:2023 Broken Object Property Level Authorization
Risk Level	HIGH

What was tested: A simple GET request was sent to the /users' endpoint without providing any form of authentication no API key, no Bearer token, no session cookie. The API was asked to return all registered user records.

What happened: The API returned a 200 OK response containing the complete records of all 10 registered users in plain JSON format. For every single user, the response included their full name, username, email address, full home address including street, suite, city and postcode, GPS coordinates (latitude and longitude), personal phone number, personal website, and full company details including name and business description.

Why this is a serious problem: In a production system, this level of data exposure without any authentication would constitute a significant data breach. An attacker could harvest the personal information of every user in the system with a single HTTP request. This type of vulnerability has been exploited in real-world breaches affecting millions of users often without the victim organization even being aware that the data was accessible.

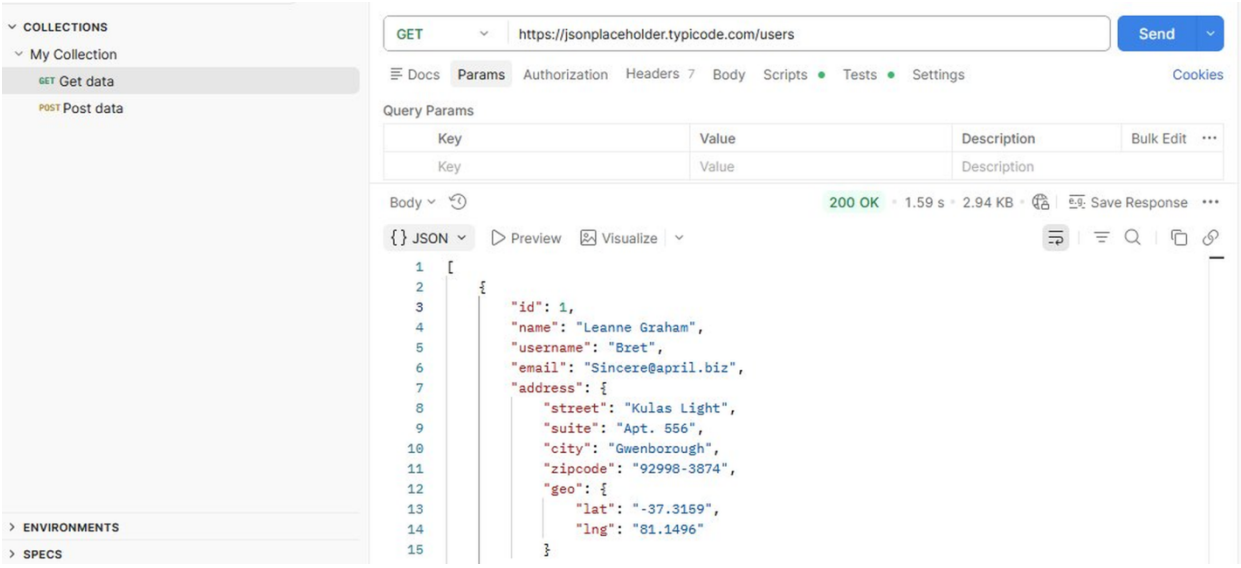

Figure 1: GET /users API returns all 10 user records including PII with no authentication (200 OK, 2.94 KB)

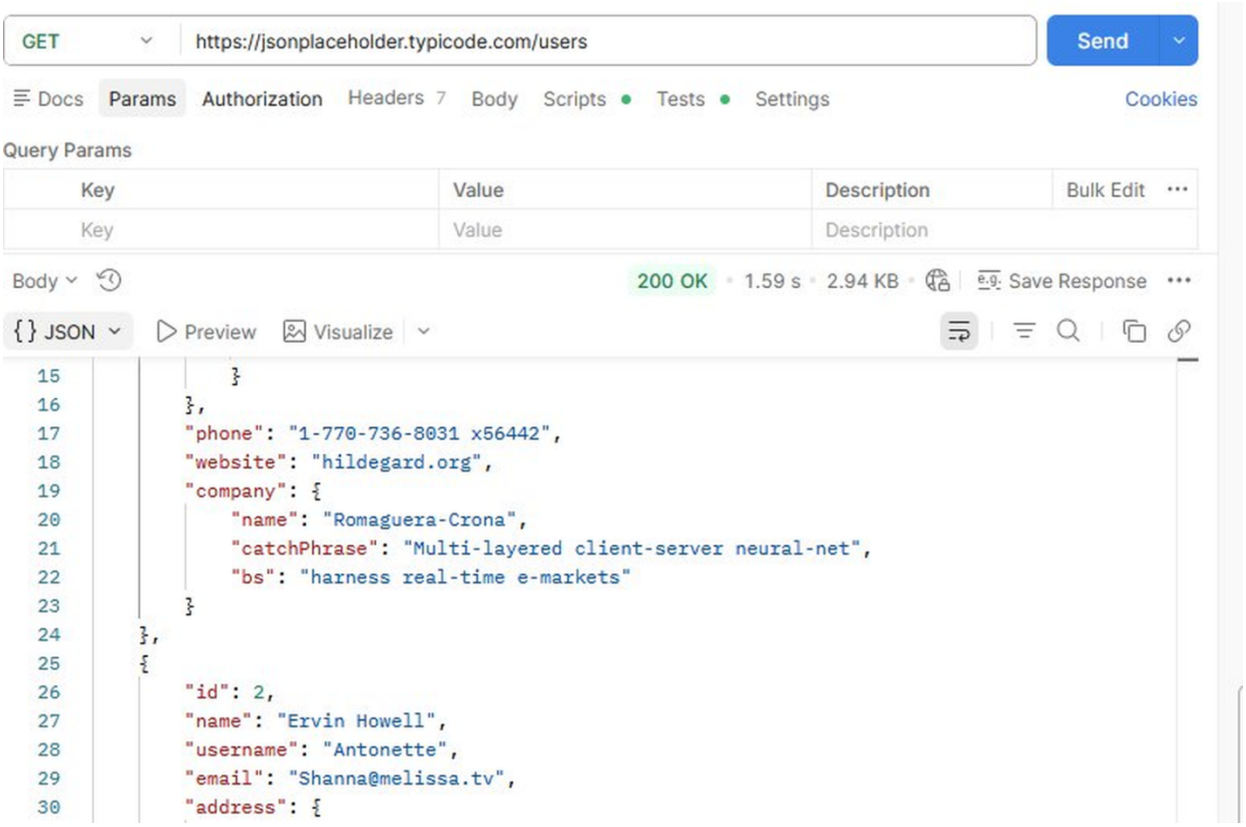

Figure 2: GET /users continued response showing multiple users' full personal details exposed

4.2 Test 2: Unauthenticated Access to Individual User PII (GET /users/1)

HTTP Method	GET
Endpoint	https://jsonplaceholder.typicode.com/users/1
Authentication	None
HTTP Status	200 OK
Response Size	1.61 KB
OWASP Category	API1:2023 Broken Object Level Authorization
Risk Level	HIGH

What was tested: A GET request was sent to retrieve the profile of a specific user by their ID number, again without any authentication. The ID value of 1 was used, but in practice any integer from 1 to 10 returns a valid user record.

What happened: The API returned the complete profile of user ID 1 (Leanne Graham) with a 200 OK response. Every field was exposed name, username, email, full address with GPS coordinates, phone number, website, and company information.

Why this is a serious problem: This vulnerability is known as Insecure Direct Object Reference (IDOR). An attacker can enumerate user IDs sequentially starting from 1 and incrementing to extract the personal data of every user in the system. No credential is needed. This technique is responsible for some of the largest data breaches in history, including breaches at major financial institutions and healthcare providers.

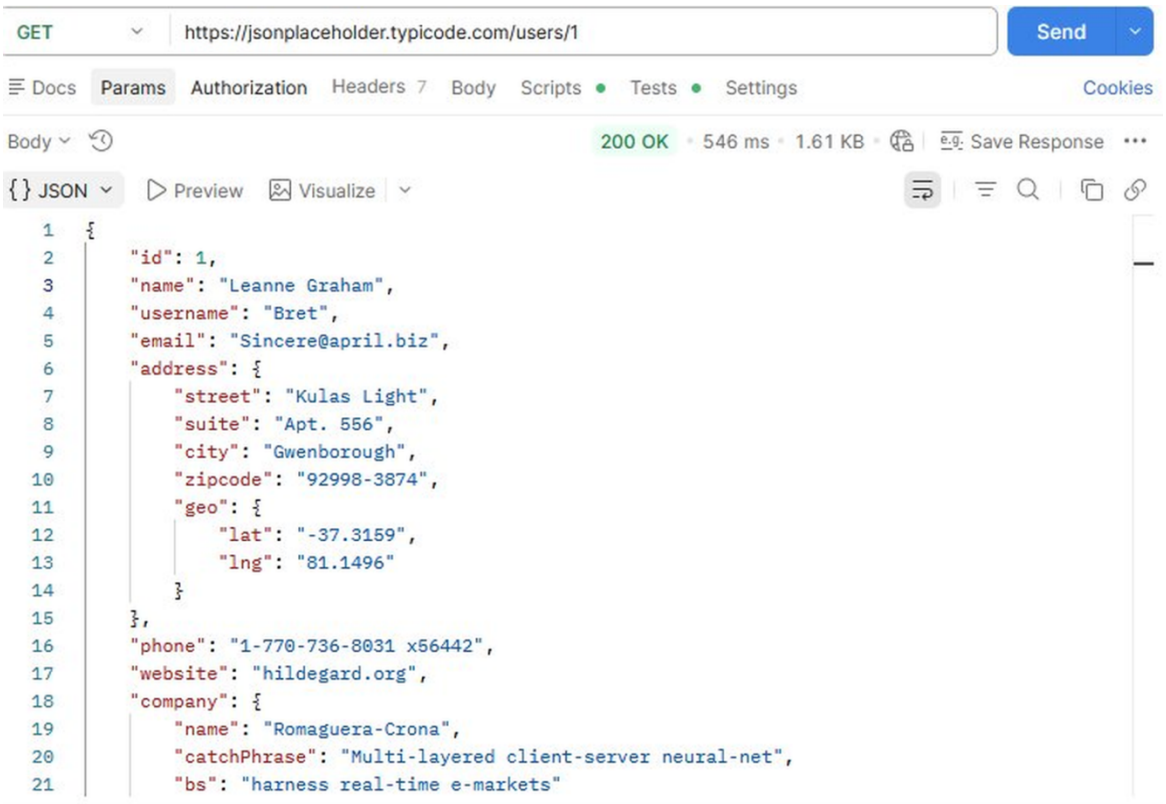

Figure 3: GET /users/1 complete PII of user Leanne Graham returned without any authentication (200 OK)

4.3 Test 3: Unauthenticated Data Creation (POST /posts)

HTTP Method	POST
Endpoint	https://jsonplaceholder.typicode.com/posts
Authentication	None
HTTP Status	201 Created
OWASP Category	API2:2023 Broken Authentication
Risk Level	HIGH

What was tested: A POST request was sent to the /posts endpoint with a JSON body containing a fabricated post titled Hacked Post. No API key, token, or any form of authentication was provided in the request headers.

What happened: The API accepted the request and responded with 201 Created confirming that the new record had been successfully created and assigned the ID 101. The API did not ask who was making the request, did not validate whether the sender had permission to create content, and did not require any form of identity verification.

Why this is a serious problem: In a real production system, the ability to create data without authentication means that any person on the internet including malicious actors can inject content, records, or transactions into the system. This can lead to data pollution, fraudulent entries, spam attacks, and in financial systems, unauthorized transactions. The API has no way of knowing whether the request came from a legitimate user or an attacker.

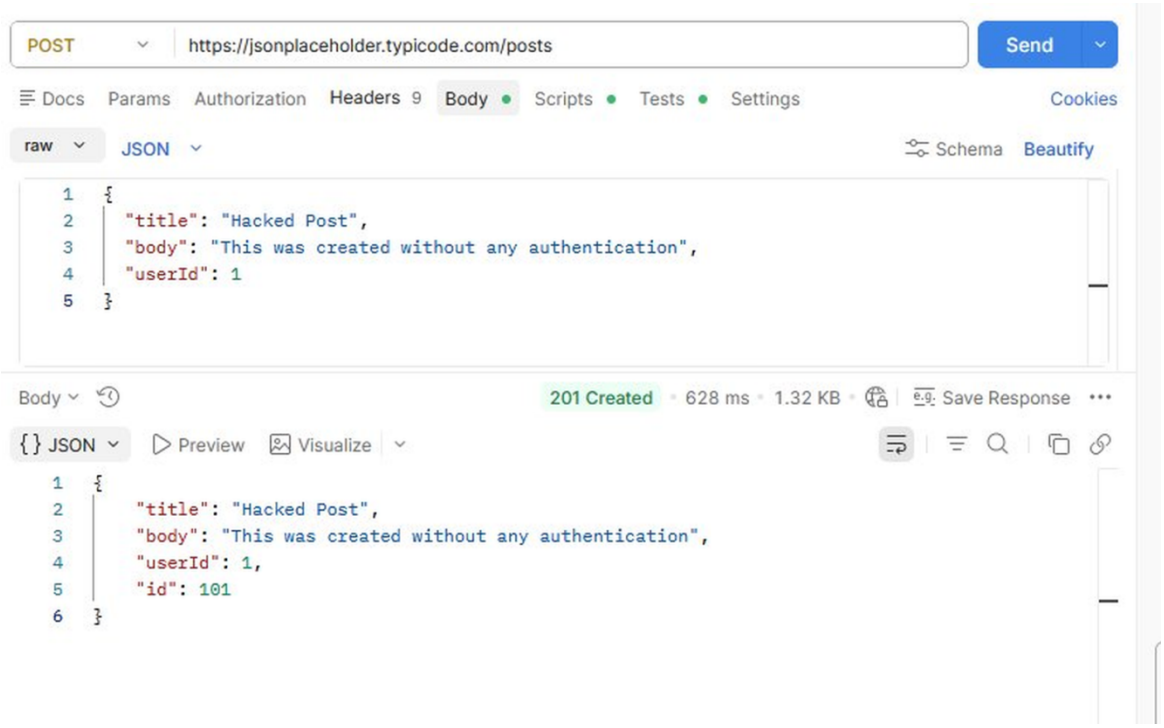

Figure 4: POST /posts unauthenticated data creation succeeded, API returned 201 Created with assigned ID 101

4.4 Test 4: Unauthenticated Data Modification (PUT /posts/1)

HTTP Method	PUT
Endpoint	https://jsonplaceholder.typicode.com/posts/1
Authentication	None
HTTP Status	200 OK
OWASP Category	API2:2023 — Broken Authentication / API5:2023 — Broken Function Level Authorization
Risk Level	HIGH

What was tested: A PUT request was sent to modify an existing post (ID 1) without providing any authentication. The request body contained new content specifically the text Unauthorized Update and This data was modified without authentication replacing the original post content.

What happened: The API accepted the modification and returned a 200 OK response confirming the update was successful. The response body showed the modified data exactly as submitted. The API did not check whether the requester owned the post, was logged in, or had any permission to modify other users' content.

Why this is a serious problem: In a real application, any authenticated user let alone an unauthenticated one should only be able to modify their own content. The ability to modify any record by simply knowing its ID is a critical authorization failure. In production environments, this type of vulnerability allows attackers to alter other users' data, tamper with records, or manipulate system state without leaving any identity trace.

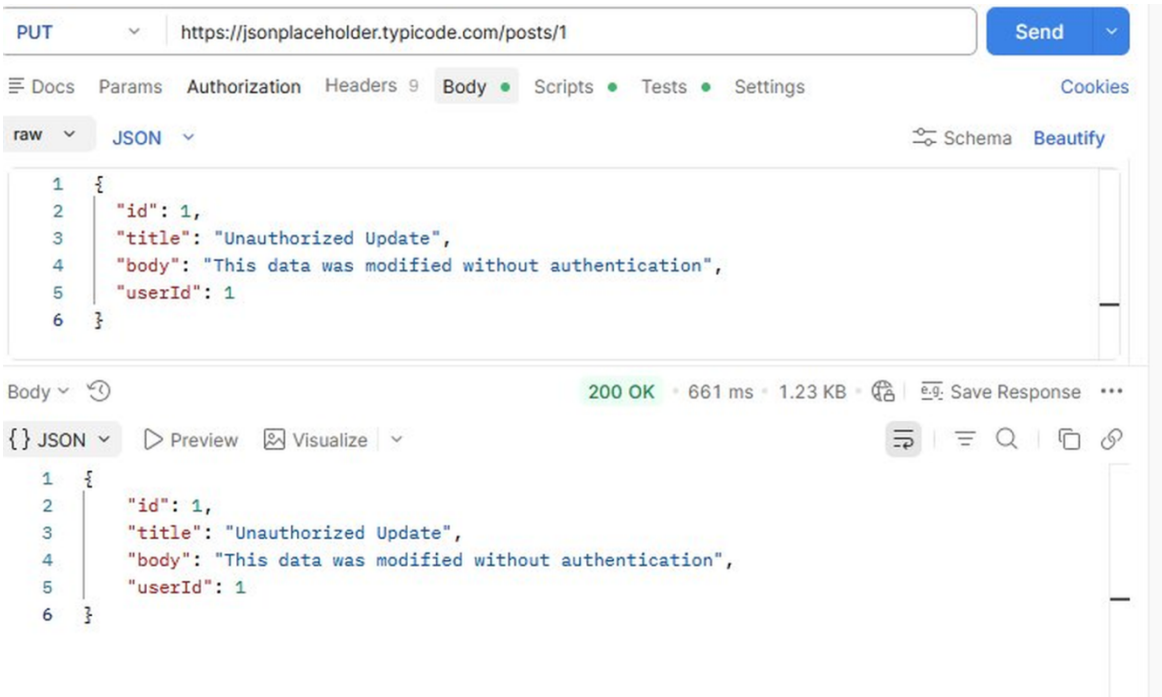

Figure 5: PUT /posts/1 existing record successfully modified without authentication (200 OK)

4.5 Test 5: Unauthenticated Data Deletion (DELETE /posts/1)

HTTP Method	DELETE
Endpoint	https://jsonplaceholder.typicode.com/posts/1
Authentication	None
HTTP Status	200 OK
OWASP Category	API5:2023 -Broken Function Level Authorization
Risk Level	CRITICAL

What was tested: A DELETE request was sent to remove post ID 1 from the system, again without any form of authentication or authorization header.

What happened: The API responded with 200 OK and an empty JSON object, confirming that the deletion request was processed successfully. The resource was accepted as deleted with no identity check, no ownership verification, and no permission validation of any kind.

Why this is a serious problem: Unauthenticated deletion is arguably the most dangerous of all the vulnerabilities identified in this assessment. In a production system, this would allow any attacker to permanently destroy data belonging to any user including critical business records, financial transactions, or user accounts. A single automated script could wipe an entire database of posts or records in seconds. Data deleted in this way may be unrecoverable if proper backup procedures are not in place.

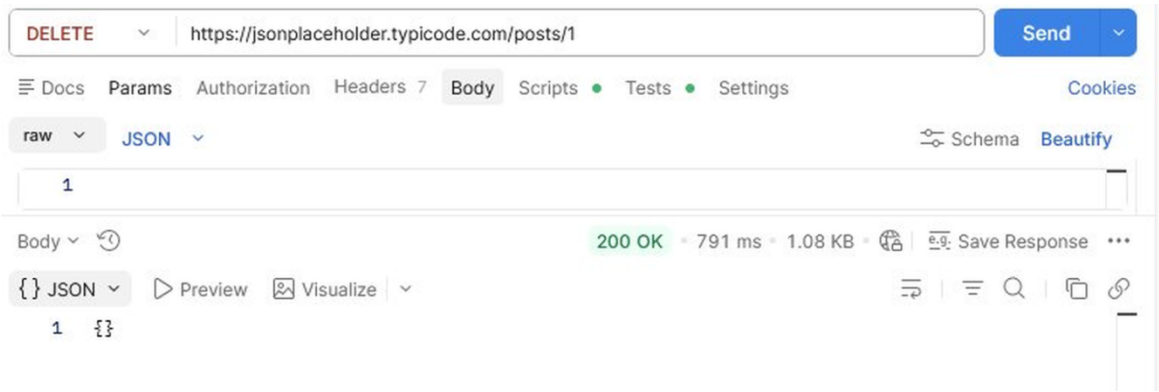

Figure 6: DELETE /posts/1 record deleted without authentication, API returned 200 OK with empty response body

4.6 Test 6: Response Header Security Analysis

HTTP Method	GET
Endpoint	https://jsonplaceholder.typicode.com/posts/1
Authentication	None
HTTP Status	200 OK
OWASP Category	API7:2023- Server Side Request Forgery / API8:2023-Security Misconfiguration
Risk Level	MEDIUM

What was tested: After sending a standard GET request, the response headers tab in Postman was inspected to evaluate whether the API implements recommended HTTP security headers.

What was found: The response headers revealed several important security observations. The positive findings include the presence of x-content-type-options: no sniff which prevents MIME-type sniffing attacks, and x-rate limit: 1000 confirming that some rate limiting is in place. The server is identified as Cloudflare, and x-powered-by: Express reveals the underlying framework.

However, several critical security headers are entirely absent from the API responses. There is no Content-Security-Policy header, which means the API provides no protection against cross-site scripting attacks in any web client consuming the API. There is no X-Frame-Options header, leaving any web application using this API vulnerable to clickjacking. The access-control-allow-credentials header is set to true, which combined with a permissive CORS policy could allow malicious websites to make authenticated cross-origin requests on behalf of users. Additionally, the x-powered-by: Express header needlessly discloses the server technology stack to potential attackers.

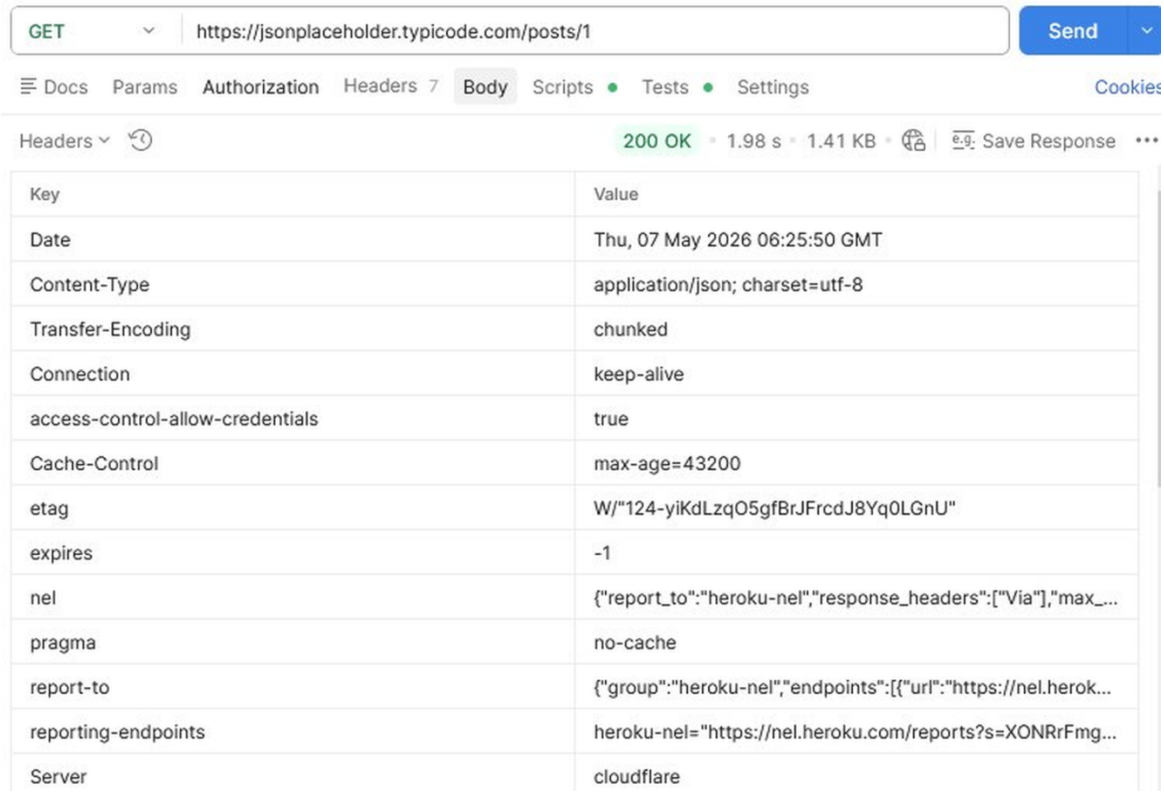

Figure 7: Postman response headers — Date, Content-Type, access-control-allow-credentials: true, Cache-Control, Server: Cloudflare

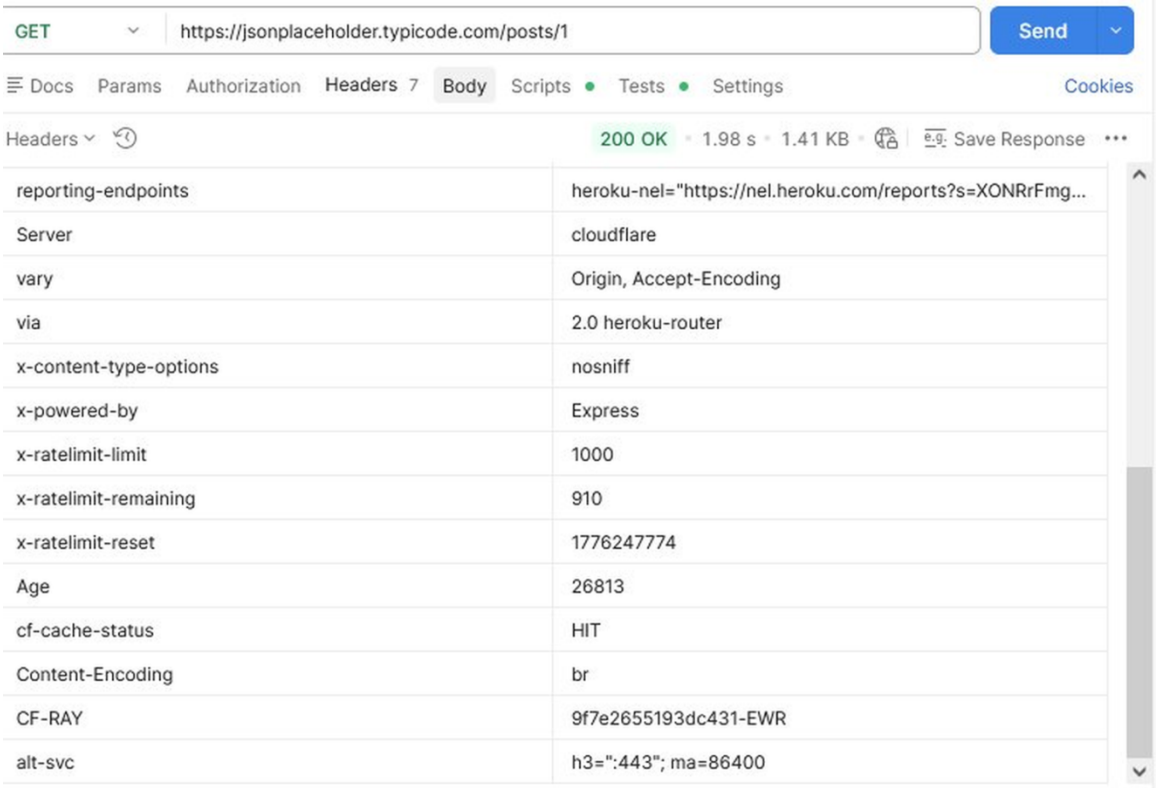

Figure 8: Postman response headers continued x-powered-by: Express disclosed, x-ratelimit-limit: 1000, missing CSP and X-Frame-Options

4.7 Test 7: Invalid Resource Handling (GET /users/999999)

HTTP Method	GET
Endpoint	https://jsonplaceholder.typicode.com/users/999999
Authentication	None
HTTP Status	404 Not Found
OWASP Category	API9:2023- Improper Inventory Management
Risk Level	INFO -Acceptable Behaviour

What was tested: A GET request was sent using a deliberately invalid user ID (999999) to evaluate how the API handles requests for resources that do not exist.

What happened: The API correctly returned a 404 Not Found response with an empty JSON body. No internal error messages, stack traces, database error details, or sensitive debugging information were exposed in the response.

Why this is a positive finding: Proper 404 error handling is a security best practice. Many poorly configured APIs return verbose error messages or stack traces when invalid inputs are provided, which can give attackers valuable intelligence about the internal structure of the system. In this case, the API behaves correctly it acknowledges the resource does not exist without leaking any information that could aid an attacker.

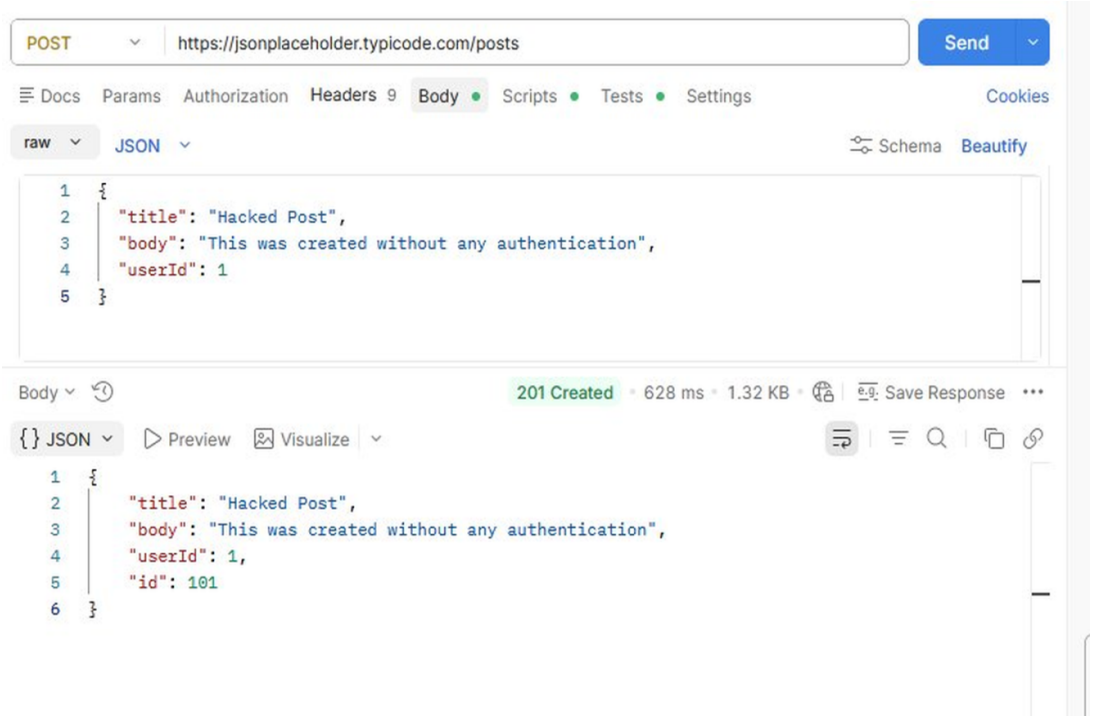

Figure 9: GET /users/999999 — API correctly returns 404 Not Found with no sensitive error information disclosed

5. SECURITY RISK FINDINGS

The table below summarises all security vulnerabilities identified during the assessment, mapped to the OWASP API Security Top 10 (2023) framework:

#	Finding	OWASP Category	Risk Level	HTTP Method / Endpoint
1	Unauthenticated Access to All User PII	API1, API3	**HIGH**	GET /users
2	IDOR- Individual User PII Exposed by ID	API1	**HIGH**	GET /users/{id}
3	Unauthenticated Data Creation	API2	**HIGH**	POST /posts
4	Unauthenticated Data Modification	API2, API5	**HIGH**	PUT /posts/{id}
5	Unauthenticated Data Deletion	API5	**CRITICAL**	DELETE /posts/{id}
6	Missing Content-Security-Policy Header	API8	**MEDIUM**	All endpoints
7	Missing X-Frame-Options Header	API8	**MEDIUM**	All endpoints
8	Overly Permissive CORS Configuration	API8	**MEDIUM**	All endpoints
9	Technology Stack Disclosure (x-powered-by)	API8	**LOW**	All endpoints
10	Weak Rate Limiting (1000 req/window)	API4	**LOW**	All endpoints
11	Proper 404 Error Handling	API9	**INFO PASS**	GET /users/999999

6. RISK SUMMARY

CRITICAL: 1	HIGH: 4	MEDIUM: 3	LOW: 2

7. BUSSINESS IMPACT ANALYSIS

Understanding the technical vulnerabilities is only half of the picture. What matters equally especially for business decision-makers and non-technical stakeholders is understanding what these vulnerabilities mean in practice. The following section translates each finding into real-world business consequences:

Vulnerability	Business Risk	Potential Consequence
No Authentication on Read Endpoints	Data privacy breach	Customer personal data exposed to anyone on the internet. Regulatory fines under GDPR, PDPA, or local data protection laws. Reputational damage and loss of customer trust.
No Authentication on Write Endpoints	Data integrity failure	Attackers can create fraudulent records, modify financial data, or delete critical business information. Loss of data integrity makes audit trails unreliable.
Unauthenticated Deletion	Catastrophic data loss	An automated attack could delete entire datasets within seconds. If proper backups do not exist, the data may be permanently lost.
Missing Security Headers	Client-side attack exposure	Web applications consuming this API are vulnerable to XSS and clickjacking attacks that could steal user sessions or trick users into performing unintended actions.
Excessive Data Exposure	Privacy violation	GPS coordinates, phone numbers, and home addresses being exposed constitutes a serious privacy risk. This data could be used for physical targeting, identity theft, or social engineering attacks.

8. REMEDIATION RECOMMENDATIONS

The following remediation actions are recommended in order of priority. Critical and High severity items should be addressed immediately before any API is deployed to a production environment:

#	Finding	Risk	Priority	Remediation Action
1	Unauthenticated Data Deletion	**CRITICAL**	Immediate	Implement mandatory authentication (JWT or OAuth 2.0) for all DELETE endpoints. Verify ownership before allowing deletion.
2	Unauthenticated Access to User PII	**HIGH**	Immediate	Protect all /users' endpoints with authentication. Return only the data the authenticated user is authorised to see.
3	Unauthenticated POST / PUT	**HIGH**	Immediate	Require valid authentication tokens on all write operations. Validate that the authenticated user owns the resource being modified.
4	IDOR Vulnerability	**HIGH**	Immediate	Implement object-level authorization checks. Never trust user-supplied IDs without verifying the requester has permission to access that specific object.
5	Missing Content-Security-Policy	**MEDIUM**	Short-term	Add a strict CSP header to all API responses: Content-Security-Policy: default-src 'self'
6	Missing X-Frame-Options	**MEDIUM**	Short-term	Add X-Frame-Options: DENY to all responses to prevent clickjacking of web clients consuming this API.
7	Overly Permissive CORS	**MEDIUM**	Short-term	Restrict access-control-allow-origin to trusted domains only. Avoid using wildcard (*) in production environments.
8	Technology Stack Disclosure	**LOW**	Long-term	Remove or suppress the x-powered-by header in Express.js: app. disables('x-powered-by')

#	Finding	Risk	Priority	Remediation Action
9	Weak Rate Limiting	**LOW**	Long-term	Reduce rate limits for sensitive endpoints such as /users. Implement per-user rate limiting in addition to global limits.

9. WHAT THIS MEANS IN SIMPLE TERMS

This section is written for non-technical readers business owners, managers, and decision-makers who need to understand the risk without needing to understand the technical details.

Think of an API like the service window at a bank. When you walk up to the window, a teller verifies your identity before giving you access to your account. They check your ID, they confirm you are the account holder, and they only show you your own account details not anyone else's.

What we found in this assessment is the equivalent of a bank where the service window has no teller. Anyone can walk up and ask for any account's details and the window will hand them over without asking a single question. Not only that, but anyone can also deposit money into any account, change the balance on any record, or delete any account entirely all without proving who they are.

In the digital world, this is exactly what is happening with the API we tested. Any person anywhere in the world with basic technical knowledge can send a simple request to this API and retrieve the personal information of every user in the system their names, home addresses, phone numbers, and GPS location. They can also create new records, modify existing ones, or delete them permanently, all without any login or permission check.

If this were a real production API used by a company, the consequences could include regulatory fines for data protection violations, lawsuits from affected customers, loss of customer trust and business reputation, financial losses from fraudulent data manipulation, and potentially criminal liability for negligent data handling.

The good news is that all of these issues are fixable. They require implementing authentication essentially adding the teller back to the service window and ensuring that every request is checked before it is fulfilled. These are well-understood engineering practices that any development team can implement.

10.LONG-TERM SECURITY RECOMMENDATIONS

Beyond the immediate fixes identified above, the following long-term security practices are recommended for any organization operating a REST API in a production environment:

#	Recommendation	Category	Detail
1	**Implement OAuth 2.0 or JWT Authentication**	Identity	All API endpoints should require a valid authentication token. Use industry-standard protocols such as OAuth 2.0 or JSON Web Tokens (JWT) to manage authentication securely.
2	**Adopt a Least-Privilege Access Model**	Authorization	Users should only be able to access and modify data that belongs to them. No user should ever have access to another user's data unless explicitly granted.
3	**Conduct Regular API Security Testing**	Testing	Schedule quarterly API security assessments using tools such as Postman, OWASP ZAP, or Burp Suite. Include API security in the CI/CD pipeline to catch vulnerabilities before deployment.
4	**Implement Comprehensive API Logging**	Monitoring	Log all API requests including the requester's identity, timestamp, endpoint accessed, and response code. Monitor logs for unusual patterns that may indicate an attack in progress.
5	**Apply the OWASP API Security Top 10**	Framework	Use the OWASP API Security Top 10 as a checklist during every API development and review cycle. This framework is updated regularly to reflect emerging threats.
6	**Implement an API Gateway**	Architecture	Deploy an API gateway to centrally enforce authentication, rate limiting, input validation, and threat detection across all API endpoints.

11.CONCLUSION

This API security risk analysis of the JSONPlaceholder REST API identified ten distinct security vulnerabilities across seven different test scenarios, including one Critical severity finding, four High severity findings, three medium severity findings, and two Low severity findings.

The most significant finding is that the API accepts all HTTP methods GET, POST, PUT, and DELETE without any form of authentication or authorization. This means that sensitive personal data including names, addresses, phone numbers, and GPS coordinates can be read by anyone, and that any person on the internet can create, modify, or permanently delete records without proving their identity or having any permission to do so.

Additionally, the API's response headers are missing several important security controls including Content-Security-Policy and X-Frame-Options, and the CORS configuration is overly permissive in a way that could enable cross-origin attacks against web clients consuming the API.

On a positive note, the API does handle invalid resource requests correctly returning a clean 404 response without disclosing any sensitive internal information and basic rate limiting is in place, though the threshold of 1,000 requests per window is insufficient to prevent a determined attacker.

The root cause of all Critical and High severity findings is the same: a complete absence of authentication and authorization controls. This is not a complex architectural problem it is a fundamental security baseline that every production API must implement. The recommended remediation is clear, well-understood, and achievable using industry-standard tools and frameworks.

Any organization deploying an API with these characteristics in a production environment would be exposing itself to significant legal, financial, and reputational risk. The findings in this report serve as a practical demonstration of why API security must be treated as a first-class concern from the earliest stages of development not as an afterthought applied after deployment.